

Exploitation d'une BD – TP2

Corrigé TP2

Les différents types de jointures

Dans cette séance, nous écrivons des requêtes utilisant différents types de jointures :

- Jointures interne (INNER JOIN, NATURAL JOIN)
- Jointure Externe (OUTER JOIN)
- Jointure croisée (CROSS JOIN).

Exercice 1 (sur feuille)

On considère les 2 tables suivantes :

Précisez ce que retourneront les requêtes suivantes :

Table1

<i>id</i>	<i>nom</i>
1	A
2	B

Table2

<i>nom</i>	<i>desc</i>
B	x
C	y
B	z

1. SELECT *
FROM Table1 T1 INNER JOIN Table2 T2
ON T1.nom=T2.nom;

	<i>id</i>	<i>nom</i>	<i>nom</i>	<i>des</i>
▶	2	B	B	x
	2	B	B	z

2. SELECT *
FROM Table1 NATURAL JOIN Table2;

	<i>nom</i>	<i>id</i>	<i>des</i>
▶	B	2	x
	B	2	z

3. SELECT *
FROM Table1 CROSS JOIN Table2;

	<i>id</i>	<i>nom</i>	<i>nom</i>	<i>des</i>
▶	1	A	B	x
	2	B	B	x
	1	A	C	y
	2	B	C	y
	1	A	B	z
	2	B	B	z

4. SELECT *
FROM Table1 T1 LEFT OUTER JOIN Table2 T2
ON T1.nom=T2.nom;

	<i>id</i>	<i>nom</i>	<i>nom</i>	<i>des</i>
▶	2	B	B	x
	2	B	B	z
	1	A	NULL	NULL

5. SELECT *
FROM Table1 T1 RIGHT OUTER JOIN Table2 T2
ON T1.nom=T2.nom;

	<i>id</i>	<i>nom</i>	<i>nom</i>	<i>des</i>
▶	2	B	B	x
	2	B	B	z
	NULL	NULL	C	y

Exercice 2 (sur feuille)

On considère les 3 tables suivantes.

Etudiants

idEtud	Nom
1	Eric
4	Lisa
6	Marc

Formations

intitulé	nbHeure
SQL	45
Java	30
HTML	25

Suivre

idEtud	intitule
1	SQL
1	Java
4	Java

Etudiants. Clé primaire : idEtud

Formations. Clé primaire : intitulé

Suivre. Clé primaire : (idEtud, intitule)

Clés étrangères : idEtud et intitule

Pour chacune des requêtes ci-dessous :

- écrivez les lignes qui s'afficheront à l'écran
- écrivez une phrase en français correspondant à ce que retourne la requête

Requête 1

SELECT *

FROM Etudiants e LEFT OUTER JOIN Suivre s

ON e.idEtud = s.idEtud

Lister tous les étudiants, avec les formations qu'ils suivent si elles existent, et des valeurs NULL sinon.

idEtud	nom	idEtud	intitule
1	Eric	1	Java
1	Eric	1	SQL
4	Liza	4	Java
6	Marc	NULL	NULL

Requête 2

SELECT *

FROM Etudiants e LEFT OUTER JOIN Suivre s

ON e.idEtud = s.idEtud

WHERE s.intitulé is NULL

Lister les étudiants qui ne suivent aucune formation.

idEtud	nom	idEtud	intitule
6	Marc	NULL	NULL

Requête 3

SELECT *

FROM Etudiants e RIGHT OUTER JOIN Suivre s

ON e.idEtud = s.idEtud

Afficher les formations suivies par au moins un étudiant, en indiquant pour chacune l'étudiant qui la suit.

idEtud	nom	idEtud	intitule
1	Eric	1	Java
1	Eric	1	SQL
4	Liza	4	Java

Requête 4

SELECT *

FROM Suivre s LEFT OUTER JOIN Formations f

ON s.intitule = f.intitule ;

Afficher les propriétés des formations suivies par au moins un étudiant.

idEtud	intitule	intitule	nombreHeure
1	Java	Java	30
1	SQL	SQL	45
4	Java	Java	30

Requête 5

```
SELECT *  
FROM Suivre s RIGHT OUTER JOIN Formations f  
ON s.intitule = f.intitule ;
```

Lister toutes les formations, avec les étudiants qui les suivent s'ils existent et des valeurs NULL sinon.

idEtud	intitule	intitule	nombreHeure
NULL	NULL	Html	25
1	Java	Java	30
4	Java	Java	30
1	SQL	SQL	45

Requête 6

```
SELECT *  
FROM Suivre s RIGHT OUTER JOIN Formations f  
ON s.intitule = f.intitule  
WHERE s.idEtud IS NULL;
```

Afficher les formations qui ne sont suivies par aucun étudiant.

idEtud	intitule	intitule	nombreHeure
NULL	NULL	Html	25

Exercice 3

Reprenons la base de données **EDTS2** qui a le même schéma que la base de données EDT :

1. Nous souhaitons faire le point sur les séances auxquelles aucune salle n'a été affectée.

a) Afin de répondre à cette question, on peut voir que les séances recherchées sont celles qui sont dans Seances mais dont l'idSeance n'est pas dans Utilise.

Quelle requête (n'utilisant pas de jointure externe) traduit le mieux cela ?

```
select * from Seances where idSeance not in  
(select idSeance from Utilise);
```

b) Une autre manière de répondre à notre question consiste à faire une jointure externe. Voici la requête suivante :

```
select *  
from Seances s left outer join Utilise u  
on s.idSeance=u.idSeance ;
```

Complétez de deux manières différentes la requête précédente de manière à répondre à la question. (335 lignes attendues)

```
select *  
from Seances s left outer join Utilise u  
on s.idSeance=u.idSeance  
where idSalle is NULL
```

```
select *  
from Seances s left outer join Utilise u  
on s.idSeance=u.idSeance  
where u.idSeance is NULL
```

2. Quelles sont les séances utilisant la salle IN0.09 ou aucune salle ?
(517 lignes attendues)

```
select *
from Seances s left outer join Utilise u
on s.idSeance=u.idSeance
where idSalle is NULL or idSalle="IN0.09";
```

3. Quelles sont les salles qui ne sont utilisées par aucune séance ?
Faites une première requête utilisant une jointure externe. Et une deuxième n'utilisant pas de jointure externe. (3 lignes attendues)

Requête 1

```
select *
from Salles s left outer join Utilise u
on s.idSalle=u.idSalle
where idSeance is NULL;
```

Requête 2

```
select idSalle from Salles
except
select idSalle from Utilise;
```

4. Affichez, pour chaque salle, le nombre de séances où elle a été utilisée. Vous afficherez le résultat par valeur croissante du nombre de séances.

```
select s.idSalle, count(idSeance)
from Salles s left outer join Utilise u
on s.idSalle=u.idSalle
group by s.idSalle
order by count(idSeance) ;
```

5. Ecrivez la requête affichant les groupes pour lesquels aucune séance n'a été planifiée. Faites une première requête utilisant une jointure externe. Et une deuxième utilisant un opérateur ensembliste. (11 lignes attendues)

Requête 1

```
select g.*
from Groupes g left outer join
Seances s
on g.idGroupe=s.idGroupe
where s.idSeance is NULL;
```

Requête 2

```
select idGroupe from Groupes
except
select idGroupe from Seances;
```

Avec not NULL

```
SELECT idGroupe
FROM Groupes
WHERE idGroupe not IN
(SELECT idGroupe
from Seances
WHERE idGroupe IS NOT
NULL);
```

6. Ecrivez la requête affichant le nombre d'heures planifiées pour chaque groupe. Les groupes pour lesquels rien n'a été planifié devraient apparaître avec un nombre d'heures égal à 0. Aperçu attendu :

idGroupe	nb_heures
S5G2	0.0000
S5G2.1	0.0000
S4G3.2	0.0000
S4	8.0000
S4G1.1	15.0000

indication: vous pourrez commencer par obtenir un résultat tel que celui-là puis utiliser IFNULL

idGroupe	nb_heures
S5G3.2	NULL
S5G1.2	NULL
S5G2	NULL
S4	8.0000
S4G1.1	15.0000

```
select g.idGroupe, IFNULL(sum(time_to_sec(timediff(s.HeureFin,s.HeureDebut)))/3600,0)
nb_heures
from Groupes g left outer join Seances s
on g.idGroupe=s.idGroupe
group by g.idGroupe
order by FNULL(sum(time_to_sec(timediff(s.HeureFin,s.HeureDebut)))/3600,0);
```

7. On souhaite avoir la liste de tous les enseignants avec le nombre de séances que chacun d'eux a eu durant le mois de juin. Écrivez la requête correspondante.
aperçu attendu :

```
select p.idProf, count(s.idSeance) nbSeances
from Professeurs p
left join (select * from Seances where month(jour)=6 ) s
on p.idProf=s.idProf
group by p.idProf;
```

8. Ecrire la requête qui permet d'afficher à la fois les enseignants qui n'assurent aucune séances ainsi que les séances n'ayant aucun professeur.

```
select p.idprof, s.idseance
from Professeurs p left outer join Seances s
on p.idprof=s.idprof
where idSeance is NULL
union
select p.idprof, s.idseance
from Professeurs p right outer join Seances s
on p.idprof=s.idprof
where p.idprof is NULL;
```